Programing for Business Lab

INS: OWAIS HAMEED

Lists & Tuples (Business Data Storage)

- Lists and tuples
- Indexing and slicing
- Business data: sales records, transaction amounts

LIST VS TUPLES

List	Tuple
Lists are mutable(can be modified).	Tuples are immutable(cannot be modified).
Iteration over lists is time-consuming.	Iterations over tuple is faster
Lists are better for performing operations, such as insertion and deletion.	Tuples are more suitable for accessing elements efficiently.
Lists consume more memory.	Tuples consumes less memory
Lists have several built-in methods.	Tuples have fewer built-in methods.
Lists are more prone to unexpected changes and errors.	Tuples, being immutable are less error prone.

WHEN to USE WHAT ?

- **Lists are used when data changes, tuples are used when data is fixed (records).**

- Example:

1. **List : daily sales**
2. **Tuple : transaction record**

Basic Syntax

- **List**

```
listname = [value1, value2, value3]
```

```
daily_sales = [1200, 1500, 1100, 1800, 2000]
print(daily_sales)
```

- **Tuples**

```
tuple_name = (value1, value2, value3)
```

```
transaction = (101, "Ali", 2500)
print(transaction)
```

Mutability Test

- **List are Mutable**

```
a = [1, 2, 4, 4, 3, 3, 3, 6, 5]

a[3] = 77
print(a)
```

- **Tuples are Immutable**

```
b = (0, 1, 2, 3)

b[0] = 4
print(b)
```

Memory Efficiency Test

```
# Creating an empty list and tuple
import sys
a = []
b = ()

a = [“programing", "For", “business"]
b = (“programing", "For", “business")

print(sys.getsizeof(a))
print(sys.getsizeof(b))
```

Concatenation

Both lists and tuples can be concatenated using the **"+" operator.**

```
# List Concatenation
a = [1, 2, 3]
b = [4, 5, 6]
print(a + b)

# Tuple Concatenation
a = (7, 8, 9)
b = (10, 11, 12)
print(a + b)
```

Slicing in list and tuples

- List [start : end]
- List [: end]
- List [start :]

```
# List  and tuple slicing
sales[1:4]
sales[:3]
sales[2:]
```

LIST FUNCTIONS

Function	Purpose
len()	count elements
sum()	total values
max()	largest value
min()	smallest value

```
sales = [1200,1500,1800]

print(len(sales))
print(sum(sales))
print(max(sales))
print(min(sales))
```

List-Specific operations

Only lists support operations that modify their contents:

- **Append:** Adds an element at the end.
- **Extend:** Merges another list.
- **Remove:** Removes the first occurrence of a value.

```
a = [1, 2, 3]

a.append(4)
a.extend([4,5, 6])
a.remove(2)
a.pop()
print(a)
```

ITERATING LIST AND TUPLES

```
transaction = (101, "Ali", 2500)

for item in transaction:
    print(item)
```

```
sales = [1200, 1500, 1800]

for i in range(len(sales)):
    print("Day", i+1, "Sales:", sales[i])
```

```
sales = [1200, 1500, 1100, 1800, 2000]

for s in sales:
    print("Daily Sales:", s)
```

TAKING INPUTS In a LIST

```
n = int(input("How many items? "))
data_list = []

for i in range(n):
    item = input(f"Enter item {i+1}: ")
    data_list.append(item)
```

TAKING INPUTS In a TUPLE

METHOD 1

```
tid = int(input("Enter Transaction ID: "))
name = input("Enter Customer Name: ")
amount = float(input("Enter Amount: "))

transaction = (tid, name, amount)

print(transaction)
```

TAKING INPUTS In a TUPLE

METHOD 2: Use Split for Multiple Inputs

```
transaction = tuple(input("Enter values separated by space: ").split())

print(transaction)
```

TAKING INPUTS In a TUPLE

METHOD 3: Creating Tuple from List Input

```
data = []

for i in range(3):
    value = input("Enter value: ")
    data.append(value)

transaction = tuple(data)

print(transaction)
```

TASK-1

Store Daily Sales Using Input

Problem:

1. Ask user how many sales to store (eg: 5)
2. Takes **that number of daily sales values from the user(e.g. user enter 5 values)**
3. Stores them in a **list**
4. Displays all sales values using a loop.

TASK-2

Problem

Take **5 sales values from the user** and display:

1. First three days sales
2. Last two days sales

TASK-3

Revenue Calculator

Problem:

1. Ask user how many sales to store (eg: 5)
2. Takes **n sales values as input**
3. Stores them in a list
4. Calculates **total revenue**

TASK-4

Problem:

1. Takes **3 customers for branch A as input**
2. Takes **3 customers for branch B as input**
3. Combines both lists using **extend()**
4. Displays the full customer list
5. Delete the last customer
6. Display again